

I want to study and practice design pattern in java lets start from singleton pattern

Great choice 👍

Design Patterns are **very important for Java backend roles**, especially with your Spring / enterprise background.

Let's start **step-by-step** with the **Singleton Pattern**.

1 What is the Singleton Pattern?

Definition (simple):

The Singleton Pattern ensures that a **class has only one instance** and provides a **global access point** to that instance.

Real-world analogy

- Configuration manager
- Database connection pool
- Cache
- Logger

You don't want multiple copies of these in memory.

2 When should you use Singleton?

Use it when:

- You need **exactly one instance** of a class
- That instance must be **shared globally**
- Creating multiple instances would cause **inconsistency or resource waste**

✗ Don't use it just because it's popular — overusing Singleton is a bad design.

3 Basic (Naive) Singleton – ❌ NOT thread-safe

java

 Copy code

```
public class Singleton {  
  
    private static Singleton instance;  
  
    private Singleton() {  
        // private constructor  
    }  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Problem

- In **multi-threaded** applications (like Spring Boot), two threads may create **two instances** ❌

4 Thread-Safe Singleton (Synchronized Method)

java

 Copy code

```
public class Singleton {  
  
    private static Singleton instance;  
  
    private Singleton() {}  
  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

```
}  
}
```

Pros

- ✅ Thread-safe
- ❌ Slower (every call is synchronized)

5 Best Practice #1: Double-Checked Locking

java

 Copy code

```
public class Singleton {  
  
    private static volatile Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

Why this is good

- Fast
- Thread-safe
- Widely used in real systems

6 Best Practice #2: Bill Pugh Singleton

java

 Copy code

```
public class Singleton {  
  
    private Singleton() {}  
  
    private static class SingletonHelper {  
        private static final Singleton INSTANCE = new Singleton();  
    }  
  
    public static Singleton getInstance() {  
        return SingletonHelper.INSTANCE;  
    }  
}
```

Why this is excellent

- ✓ Lazy initialization
- ✓ Thread-safe
- ✓ No synchronization overhead
- ✓ Clean and readable

👉 This is the most recommended Singleton in Java interviews

7 Best Practice #3: Enum Singleton (Most robust) 💡

java

 Copy code

```
public enum Singleton {  
    INSTANCE;  
  
    public void doSomething() {  
        System.out.println("Singleton using enum");  
    }  
}
```

Why Enum Singleton?

- Thread-safe by default

- Protects against **serialization** and **reflection attacks**
- Used in **high-quality libraries**

👉 Many senior Java engineers consider this the **best**.

8 How Singleton appears in Spring Boot (Important for you)

In Spring:

java

📋 Copy code

```
@Service
public class UserService {
}
```

- By default, Spring beans are **Singleton**
- Spring manages the lifecycle for you

java

📋 Copy code

```
@Scope("singleton")
```

9 Common Interview Questions (You MUST know)

Q1: Is Singleton anti-pattern?

👉 Not always. Overuse is bad, but it's valid for shared resources.

Q2: How do you break Singleton?

- Reflection
- Serialization
- Cloning

Q3: How to protect?

- Use **Enum Singleton**
 - Throw exception in constructor
 - Override `readResolve()`
-

1 0 Practice Task for You 🗨️

Try this:

1. Implement **Bill Pugh Singleton**
2. Write a `main` method and prove:
 - Two calls return the **same object**
3. Then convert it to **Enum Singleton**

If you want, next we can continue with:

- **Factory Pattern**
- **Builder Pattern**
- **Strategy Pattern**
- **State Pattern** (you already touched this in CBHI 😊)

👉 Tell me:

Do you want to go next with **Factory Pattern** or deepen **Singleton** with tests & edge cases?

📄 👍 💬 ↗️ ↺️ ⋮ Compare with Claude ▾

is creationl

Yes ✅ Singleton is a *Creational Design Pattern*.

Why Singleton is Creational?

Creational patterns deal with:

- Object creation
- How objects are instantiated
- Controlling the creation process

👉 Singleton controls **how many objects** are created

👉 It guarantees **only one instance**

That's exactly why it belongs to the **Creational** category.

Creational Design Patterns in Java

Here's the full list you should remember (very common interview question):

Pattern	Purpose
Singleton	Ensure only one instance exists
Factory Method	Create objects without specifying exact class
Abstract Factory	Create families of related objects
Builder	Construct complex objects step by step
Prototype	Clone existing objects

📌 **Mnemonic: S-FABP**

(Singleton, Factory, Abstract Factory, Builder, Prototype)

How interviewers usually ask

? "Which category does Singleton belong to?"

✅ **Creational Design Pattern**

? "Why not Structural?"

👉 Because it doesn't deal with object composition

👉 It controls **object creation**, not structure

One-line exam answer 🗨️

Singleton is a creational design pattern because it restricts object creation to a single instance and provides a global access point to it.

Next step recommendation for you (based on your backend experience 🧑💻):

➡️ **Factory Pattern** (very common in Spring & real projects)

or

 **Builder Pattern** (used heavily with DTOs, immutability)

 Which one do you want to learn next?

     ... Compare with Claude ▾